AI-Powered Payment Fraud Detection and Optimization System

Deepa Mathew K.[1], Krishna K G[1], Deepa S G[1], and Deepak Yadav[1]

Department of Artificial Intelligence and Machine Learning
Dayananda Sagar Academy of Technology and Management
Bangalore, India
deepa-aiml@gmail.com
krishnakg1205@gmail.com
deepasg2110@gmail.com
deepakrises2002@gmail.com

Abstract. Nowadays booming digital financial services have led to a surge in transaction volume and payment fraud. Such static threshold and rule-based rules are ill-suited for adapting to dynamic attacks and as a result, these systems tend to suffer from high false positive rates and substantial financial damage. In this paper, we first introduce a machine learning-based payment fraud detection system, which employs cost-aware models to optimize both precision and cost-standard goals under strict business constraints. We use the PaySim dataset which simulates mobile money transactions in large-scale with realistic class imbalance (fraud rate approximately 0.13%), the proposed framework integrates cost-sensitive learning, graph-based relational features as well as economic cost-optimization mechanism for threshold selection. The last model yields a recall and precision of fraud class at 93.67% and precision of 66.86%, with an AUC-ROC of 0.9841. Under an asymmetric cost model, a cost-sensitive optimal point 0.0892 minimizes total expected financial loss. Graph-derived features enable near graph neural network performance while preserving sub-50 ms median inference latency. Results demonstrate that hybrid approaches combining gradient boosting, graph-based features, and explicit cost modeling deliver substantial operational and economic benefits for payment fraud detection.

Keywords: Fraud detection · XGBoost · Graph-based features · Cost sensitive learning · PaySim · Real-time systems

1 Introduction

Money handling changed completely because of digital tools. Through phones or online systems people now send funds quickly around the world. Yet faster access opens doors to clever scams too. Criminals find ways to exploit weak spots hidden in complex networks. Losses add up past hundreds of billions every year.include [1, 2] More activity means more chances for theft to slip through. Every move is accompanied by the unknown dangers.

Modern-day fraudsters apply sophisticated techniques such as account hijacking, rewiring, operating structured fraud groups, and laundering money schemes. They are silent at first but their activity spans time connected together by obvious links - smuggling through more advanced detection methods. Systems built on fixed regulations tend to be based upon existing boundaries, such as prohibiting amounts over $10,000 to be paid to some nations, but still, clever attackers get acquainted with these lines and avoid them. Since they unable to adapt, tricks of a new kind that are introduced almost unnoticed remain so until it is too late.

Machine is one of the primary facilitators of next-generation fraud detection learning (ML). Trained models of transaction data. Past can detect minor indications of fraud and detect complicated non-linear decision boundaries. Random forest, Gradient ensemble methods, LightGBM, Boosting, and XGBoost are commonly applied in the industry and have been used. shown to be performing well on tabular fraud datasets [4–7]. Three major there are still impediments on the way to the practical implementation of ML-driven, fraud detection, though:

Extreme discrepancy in classes: Fraud is underrepresented in a real-life setting. than 0.2% of transactions. By assuming that all the transactions will be valid. naive models with even high scores on almost all cases of fraud, even in the face of very large probabilities of success. overall accuracy.

Changing frauds when users change: Static thresholds and concept drift. rules, so strict habits begin to crumble at any time. When models slip, nobody notices - until losses pile up.

False asymmetry: False positives (required investigations) are significantly lower. costlier than false negatives (false alarms) in fraud prevention. Optimising only a statistical measure, e.g. accuracy or AUC, can lead to less. greater than optimum business performance.

Precise modelling of the transaction network, where the accounts are nodes and edges are transactions and have been demonstrated in new graph neural network (GNN). studies to significantly increase the identification of organized fraud groups and multi. hop patterns [8–11]. But due to their computational cost and the scaling dynamics graph scaling Dynamic graphs can be difficult to scale complex GNN-based solutions. to execute at very strict latency requirements.

1.1 Research Gaps and Motivation

Although there have been advancements in the ML- and graph-based fraud detection, a number of research gaps exist. remain:

Gap 1 - Hybrid graph based tabular modeling: either of the two many works. use only pure tabular models (XGBoost, LightGBM) or complete GNN archetypes. tures. Minor effort is made on hybrid systems which are a graph rea approximation. soning by using graph features that are engineered, and still keeping the simplicity and benefits of trees ensembles in deployment. Detection and Optimization of Payment Fraud 3.

Gap 2 - Explicit cost-conscious optimization: This is at large fraction of the literature maximizes pure statistical characteristics like accuracy, F1-score, or.

Without a direct modeling of asymmetric fraud and investigation costs, AUC. at decision time.

Gap 3 - Strictly latency real-time deployment: GNN-based. methods tend to be off-line or batch based. There is a lack of end-to- graph-aware modelling with mi-end system studies showing graph-aware modelling. croservices designs with sub-100 ms real time performance.

Gap 4 - Combined Detection performance and business. impact: Few works provide the detailed metrics of classification along with the reports of the metrics of the correction. respondent economic effect in realistic cost assumptions.

1.2 Contributions

This work addresses critical limitations in existing payment fraud detection systems that rely on static thresholds and accuracy-driven evaluation. The main contributions are as follows:

- We formulate payment fraud detection as a cost-sensitive decision problem and demonstrate that threshold optimization based on financial loss leads to substantially improved operational outcomes compared to accuracy-centric models.
- We introduce an adaptive fraud monitoring framework that integrates Page-Hinkley drift detection with supervised learning to address non-stationary transaction streams, a limitation largely overlooked in prior studies.
- We provide a comprehensive empirical evaluation on the PaySim dataset, including classification performance, economic cost analysis, and feature importance, offering a more deployment-oriented assessment than existing benchmarks.

2 Related Work

2.1 Traditional and Machine Learning-Based Fraud Detection

It would then require an aspect of bending hard and fast rules which had to be instituted to identify any type of fraud and this would have been enhanced by the specialists who coded the red flags of the past cases [3]. They are not quite hard to implement and they tend to bump against each other as they become strained, require continual servicing and fail to work in the existence of new pattern of frauds.

The techniques of logistic regression, decision trees, and Random Forests have greatly disseminated with the invention of machine learning, however, notions like inverse probability weighting have not been extensively adopted yet (lucas2020credit). In the case of the high skew of the information available about the fraud detection, tree-based models can be easily tempted to the necessity to offer such auxiliary means as rebalancing, classification weight distribution, and special remedial action to discover their rare cases.

The latest models of structured tabular data are gradient boosting models, in particular, XGBoost and LightGBM, and can be used in both fraud detection and in other processes, where structured tabular data is an input. The main innovations introduced by XGBoost are such as regularization to eliminate overfitting, parallel training, strong support of missing values, high-order gradient estimations of the tree dividing, etc. In addition, in situations of high imbalance in task, XGBoost can also be utilized since it has a scale of the classes, scaleposweight, that can be used to scale the classes specifically. Recent studies that use XGBoost on advanced sampling and Bayesian hyperparameter optimization procedures showed satisfactory results on credit card fraud models [6, 7]. Deep reinforcement learning based fraud detectors have also been tested on transactional data and have achieved competitive validation performance results [5].

2.2 Graph Neural Networks for Fraud Detection

Graphs Financial transactions Financing graph are natural graphs, where nodes represent accounts, devices, or merchants, and the edges represent money transfers or interaction. The fraud has been likened many times to appear as certain normal network constructions, including tightly knit conlinked rings of colluding records, chain links of money laundering, or dramatic alterations in local connection which is indicative of account takeover.

Graph neural networks have been revised and used extensively on such problems [8, 9]. Such architectures are GraphSAGE, and Graph Attention Networks. A depiction of the learn nodes (GAT) is what relies on the sum of the weights of the neighbors, facilitates the identification of suspicious neighborhoods and multi hop relationships [11]. The recent papers further discuss the temporal and causal variations of GNNs to dynamic transaction graphs. It has causal and temporal development, between interactions [12, 13]. Hybrid graph augmented RNN and hybrid GNN. Two or more frauds have also been proposed to be customized through rings and transactions [10].

But in practice in real-time payment systems, GNNs are difficult to applying. Incremental computation of large scale dynamic graphs as a multi hop execution, maintenance and update of large scale dynamic graphs, binding, and relaying of messages, may be prohibitively expensive sive. This has encouraged a hybrid method in which features obtained through the use of graphs (e.g., degree, centrality measure, degree of motifs) are calculated in advance and inputted into more conregular algorithms, such as XGBoost [7].

2.3 Real-Time Systems, Concept Drift, and Explainability

As a standard deployment pattern, live detection systems of current frauds frequently require stringent latency demands, of less than 100 ms/trans, and of thousands of transactions per second. ference service and application logic, have been a defacto deployment pattern. Others, such as FastAPI, provide entire

asynchronous reusable endpoints which are appropriate in high-throughput ML serving.

This is even more complicated considering the concept drifts, since the direction of the fraud alteration is depending on the adaptive competitors, user populations dynamics and new rules. Change in data distribution, or model behavior is monitored using drift detection algorithms, e.g. ADWIN [14], KSWIN [16] and Page-Hinkley [15]. The Page-Hinkley test is especially computationally inexpensive, and, more so, efficient when streaming.

Lastly, transparency and explainable AI (XAI) are mandatory in the financial service laws and regulations [17, 18]. SHAP and LIME also possess instance-level exlocal explainability which can be used on graph-based models by planations and more recent works. Combination of feature-level significance and dashboards understandable make faith and aid human researchers to make decisions.

Finally, since the laws and regulations of the financial services involve explainable and transparent AI (XAI) [18, 17]. SHAP and LIME are locally explainable, and instance-level explainability as well as explainability to graph-based models are also offered recently. Integrating feature-level significance and interpretable dashboards facilitates trust and helps human investigators to make decisions.

3 Problem Formulation

This part makes the task of detecting fraud a binary classification problem with economic cost implications.

3.1 Data and Labels

A transaction dataset D consists of feature vectors $\mathbf{x}_i$ and binary labels $y_i \in \{0, 1\}$, where $y_i = 1$ denotes fraud and $y_i = 0$ denotes a legitimate transaction. The PaySim dataset is highly imbalanced, with $\Pr(y = 1) \approx 0.0013$ and $\Pr(y = 0) \approx 0.9987$. The objective is to train a model f_θ that estimates the fraud probability p_i for each transaction. A decision threshold τ is then applied to classify transactions as fraudulent if $p_i \geq \tau$, and legitimate otherwise.

3.2 Cost Model

In practical fraud detection, the costs associated with errors are highly asymmetric. Let:

- C_{FN}: average cost of a false negative (missed fraud),
- C_{FP}: average cost of a false positive (unnecessary investigation).

Given a set of predictions, let $N_{\mathrm{FN}}(\tau)$ and $N_{\mathrm{FP}}(\tau)$ denote the number of false negatives and false positives at threshold τ. The total cost at threshold τ is modeled as

$$\mathrm{Cost}(\tau) = N_{\mathrm{FN}}(\tau)\, C_{\mathrm{FN}} + N_{\mathrm{FP}}(\tau)\, C_{\mathrm{FP}}. \tag{1}$$

In this work, empirically reasonable values of $C_{\mathrm{FN}} = \$1000$ and $C_{\mathrm{FP}} = \$25$ are adopted, reflecting that missing a fraud is approximately $40\times$ more expensive than investigating a legitimate transaction. The goal is not merely to maximize a statistical metric such as AUC, but rather to choose $\tau^\star$ that minimizes expected total cost:

$$\tau^\star = \arg\min_{\tau\in[0,1]} \mathrm{Cost}(\tau). \tag{2}$$

3.3 Class Imbalance in Learning Objective

Cost-sensitive weighting is used to combat class imbalance during training. Given N_0 (legitimate) and N_1 (fraud), the weight of the positive class is

$$w_{\mathrm{pos}} = \frac{N_0}{N_1}. \tag{3}$$

XGBoost has this as a direct option through the `scale_pos_weight` parameter. The loss is regularized and the weighted examples are minimized using the model:

$$L(\theta) = \sum_{i=1}^{N} w_i\, \ell\big(y_i, f_\theta(x_i)\big) + \Omega(\theta), \tag{4}$$

where $w_i = w_{\mathrm{pos}}$ in the case of fraud samples and $w_i = 1$ in the case of legitimate samples. $\Omega(\theta)$ represents the regularization terms (i.e., tree) and the binary logistic loss.

4 System Architecture

4.1 Overview

The suggested system is based on three-tier microservices architecture that is appropriate to be deployed in real time:

This system will be developed based on the implementation of three distinct levels, or microservices. The former is a user-friendly interface at the frontend that will be utilized in visualization and monitoring; the frontend user-friendly interface will be created using React.js. The second is a server which is a backend node server to process business logic; the backend node server will be built using express. Lastly the third microservice will be the one, which deploys the XGBoost model and draws inferences out of it; it will be written in Python and Fast API.

4.2 Data Flow

The received transaction requests are processed using a multi-layered architecture designed for scalability and fault isolation. Transaction data are transmitted to the backend through a REST interface in JSON format, where schema validation is performed. Contextual information stored in MongoDB, Redis, and

Table 1. PaySim Dataset Statistics

Metric	Value
Total transactions	6,362,620
Fraudulent transactions	8,213 (0.13%)
Legitimate transactions	6,354,407 (99.87%)
Time span	30 days (744 hours)
Transaction types	5
Average transaction amount	$179,861

Neo4j is then added to each transaction, producing a feature-ready input that is sent to the machine learning service.

The engineered features are calculated by the ML service and the XGBoost classifier is invoked using the optimized decision threshold t [4], which is obtained on the basis of the cost-sensitive optimization process. The model generates both a fraud probability score and a binary classification decision.

The backend logs the prediction results, updates the relevant databases, and relays the final decision to the user-facing components. Fraud alerts are provided in real time through WebSocket-based notification channels.

To provide robustness in the presence of non-stationary transaction streams, a drift detection module periodically monitors the distribution of model predictions according to the Page-Hinkley test [14, 15]. This design enables independent scaling of the machine learning components, isolates potential model failures, and permits heterogeneous technology stacks to exist across system layers.

5 Dataset and Feature Engineering

5.1 PaySim Dataset

The PaySim data is an extensively applied synthetic data simulating mobile. transactions in monies that are based on financial records. Table 1 summarizes key statistics.

The extreme ratio of classes reproduces the real life fraud detection situations providing a realistic testbed to cost-sensitive modeling.

5.2 Preprocessing

First, Raw PaySim data is cleaned by dropping irrelevant identifiers . PaySim data is first cleaned by dropping irrelevant identifiers (e.g., raw) account IDs that are subsequently used to construct graphs), eliminating missing values and deduplicating close records. Mathematical quantities e.g. step, Columns of amount, and balance-related are type-casted to the corresponding numeric types.

Label-encoded are the categorical types of transaction (e.g. CASH OUT, TRANSFER). into integer values. The step is used to derive the hour and day temporal features. variable, that is, a time step expressed in hours.

5.3 Engineered Features

The last model involves a set of features of about 45 features, which are classified into several groups:

- **Original features:** transaction step, type of payment, amount, origin and destination balances.
- **Temporal:** hour of day, day index, night indicator, weekend indicator, and the intervals of transaction activities.
- **Balance consistency features:** difference of expectations and actual post-transactions balances of both the origin and destination accounts, which reflect irregularity in updates of balance.
- **Statistical characteristics:** Z-score of amounts of transactions at the account level with relative values to historical averages, anomalous spending.
- **Graph related characteristics:** out-degree and in-degree of origin and destination accounts, and PageRank scores of centrality, calculated on a directed transaction graph whose weights are the transaction amounts.

NetworkX is used to calculate graph-based features offline, resulting in a directed digraph $G = (V, E)$ where V is a set of nodes indicating accounts, and E is a set of edges indicating transactions. The out-degree $\deg^+(v)$ and in-degree $\deg^-(v)$ of each account node $v \in V$ and PageRank score $PR(v)$ are computed and then inverted to $\deg^-(v)$ and PageRank score $PR(v)$. Individual transaction records have features like orig degree, dest degree, orig pagerank, and dest pagerank. It is in this way that XGBoost is able to leverage the relational patterns in the transaction network without the runtime cost of repeated traversals of graph or

6 Model and Training Procedure

6.1 XGBoost Configuration

The main detection model is an XGBoost classifier with a histogram-set up construction of trees and acceleration by the use of the GPUs. The important hyperparameters are:

- objective: binary:logistic
- tree method: hist
- max depth: 8
- learning rate: 0.1
- n estimators: 200
- subsample: 0.8
- colsample bytree: 0.8
- scale pos weight: N_0/N_1 (valid to fraud samples ratio)
- reg alpha: 0.1
- reg lambda: 1.0
- eval metric: logloss

Early termination with a patient of 20 rounds is employed on validation loss. to avoid overfitting.

Table 2. Chronological Train–Validation–Test Split

Subset	Proportion of Transactions
Training	70%
Validation	15%
Test	15%

6.2 Train–Validation–Test Split

The data is divided into training and validation and testing set on 70:15:15. ratio, 70% of data is training, 15% validation and 15%other for testing. The summary of sample distribution in these subsets is presented in Table 2.

6.3 Threshold Optimization

Upon training, the model predicts the probability of fraud on the test set, p_i. A series of candidate thresholds $\{\tau_k\}$ in the range $[0.01, 0.99]$ with a step size of 0.01 are tested. For each τ_k, the confusion matrix entries $(\mathrm{TN}, \mathrm{FP}, \mathrm{FN}, \mathrm{TP})$ are computed, and the total cost is calculated according to (1). The optimal threshold is then obtained as in (2).

In the experimental setup, the resulting optimal threshold is $\tau^\star = 0.0892$, which achieves a favorable trade-off between high recall and controlled false positive rate, while minimizing total estimated cost.

7 Experimental Results

7.1 Evaluation Metrics

Standard classification metrics are used, including:

- Precision, recall, and F1-score for both classes, with particular emphasis on the fraud class.
- Overall accuracy.
- Area under the ROC curve (AUC-ROC).
- Business cost metrics based on C_{FN} and C_{FP} as defined in (1).

All reported metrics in this section are computed on the held-out test set unless stated otherwise.

7.2 Classification Performance

Table 3 reports key performance metrics on the test set at the optimal threshold $\tau^\star$.

The model has a fraud-class recall of 93.67 which means that a majority of the frauds- Fraudulent transactions are identified. The precision of the fraction

Table 3. Classification Performance at Optimal Threshold $\tau^\star = 0.0892$ (Test Set)

Class	Precision	Recall	F1-score
Legitimate	0.9999	0.9989	0.9994
Fraud	0.6686	0.9367	0.7803
Overall Accuracy		0.9984	
AUC-ROC		0.9841	

Table 4. Threshold vs. Total Cost (Test Set)

τ	FN	FP	Recall	Precision	Cost ($)
0.01	12	1,847	0.9903	0.3845	58,175
0.05	45	782	0.9635	0.6028	64,550
0.0892	78	355	0.9367	0.6686	86,875
0.15	124	198	0.8994	0.8538	128,950
0.30	245	67	0.8011	0.9452	246,675
0.50	412	23	0.6656	0.9805	412,575

of the frauds of 66.86% shows that approximately two-thirds of flagged transactions are in fact fraudulent, and are, in this case, representative. A significant enhancement of the normal rule-based systems, which tend to be characterized by many efficiencies in investigation of less than 30%.

Measurements of validation monitored in training (not tabulated here) are close to. test performance, which means that in (4) such there is early stopping and regularization erroneously avoid overfitting.

7.3 Business Impact and Cost Analysis

Table 4 presents the relationship between threshold choice, error counts, and total cost, assuming $C_{\mathrm{FN}} = \$1000$ and $C_{\mathrm{FP}} = \$25$.

Low thresholds yield extremely high recall rates but a big number of false positives and hence high costs of investigations. Low thresholds minimize false positives and fast track the amount of frauds missed, making the fraud losses go up. The optimal threshold, which is denoted by 0.0892 is that which minimizes the overall cost among the values considered. The optimized threshold has a reduction in the total cost that is an order of magnitude lower than a moderate threshold of 0.50 that is near the normative default.

7.4 Key Performance Indicators

The high-level indicators under the optimum operating point include the following served:

- **Frauds identified:** Some 1,154 fraudulent transactions, correspond saving to about $1.15 million in lost dockets.

Table 5. Top 10 Features by Importance

Feature	Importance Score
`amount`	0.2847
`orig_balance_diff`	0.1523
`dest_balance_diff`	0.1401
`orig_degree`	0.0892
`type_encoded`	0.0765
`amount_zscore`	0.0654
`dest_degree`	0.0598
`step_activity`	0.0523
`orig_pagerank`	0.0412
`hour`	0.0385

- **Frauds missed:** 78 frauds were missed worth an approximated 78,000 in losses.
- **False alarms:** 355 valid transactions flagged, which is approximately costing about $8,875 investigation overhead.
- **Investigation efficiency:** Approximately 76.5 per cent of investigations are true fraud, which is a dramatic enhancement of the traditional systems 10%–30% efficiency.
- **Total cost:** it is about 86,875, the sum of the losses as a result of fraud and investigation costs.

7.5 Feature Importance

The importance measures that are built-in in XGBoost are used to show that both traditional features are important and graph based features play an important role in detecting performance. Table 5 lists the top ten features by importance score.

Balance consistency features (`orig_balance_diff`, `dest_balance_diff`) and transaction amount dominate importance, as expected. Importantly, graph-based features (`orig_degree`, `dest_degree`, `orig_pagerank`) collectively account for a substantial fraction of total importance, highlighting the value of relational signals in distinguishing fraudulent behavior.

7.6 Comparison with Existing Work

Prior studies have reported strong fraud detection performance using both tree-based models and deep reinforcement learning. For example, Tayebi and El Kafhali [6] report an AUC of 0.9879 with a fraud-class recall of 0.8740 on a real-world credit card transaction dataset, while Qayoom et al. [5] achieve a validation accuracy of 97.10% using a deep reinforcement learning framework.

Although direct numerical comparison across datasets is not strictly feasible due to differences in data characteristics and fraud prevalence, the proposed

model achieves a competitive AUC-ROC of 0.9841 and a high fraud-class recall of 0.9367 on the highly imbalanced PaySim mobile money dataset, where the fraud rate is only 0.13%. This indicates that the proposed cost-sensitive learning framework is effective even under extreme class imbalance.

Overall, the results suggest that the proposed approach is competitive with existing machine learning and reinforcement learning-based fraud detection systems, while offering improved recall and explicit cost-awareness, which are critical for real-world deployment.

7.7 Drift Detection and Stability

A Page-Hinkley-based drift detector monitors the distribution of model prediction scores over a sliding window. Let x_t denote a monitored statistic (e.g., mean prediction probability) at time t, and μ_t be the running mean. The Page-Hinkley cumulative statistic is defined as

$$m_T = \sum_{t=1}^{T} \big(x_t - \mu_t - \delta\big), \quad M_T = \min_{1 \le s \le T} m_s, \tag{5}$$

where δ is a small tolerance parameter. A drift is signaled when

$$m_T - M_T > \lambda, \tag{6}$$

for a given threshold λ.

On the test period considered, the Page-Hinkley statistic remains below the configured threshold, indicating stable behavior with no detected drift. In a production setting, a detected drift would trigger retraining or threshold re-optimization.

8 Discussion

8.1 Effectiveness of Graph-Based Features

The experimental findings prove that graph-based features, which are explicitly engineered, enhance fraud detection performance significantly. Although the system does not use full-fledged GNNs, degree and PageRank features enable the XGBoost model to capture the structure of the transaction network, e.g., highly connected hubs and suspicious central nodes.

This is a good tradeoff: much of the relational modeling strength can be retained using this hybrid model. Graph based approaches possess the modeling capacity and ensure inference is fast and scalable. Graph features can be re-calculated off-line or at moderate effort and re-calculated at an incremental rate periodically, thus the technique is suitable when the throughput is large.

8.2 Latency vs. Accuracy Trade-off

Timeliness is a significant limitation with real-time payment systems. The histogram XGBoost based XGBoost is a generalized implementation with and a small number of GPUs. Trees have median latency of inference below 50ms on standard hardware, better in usual production needs.

More complicated models, such as deep neural networks or multi-graph neural networks, hop message message relay, might potentially provide marginal accuracy benefits but would increase latency and complexity of operation as well. The findings reveal that the proposed system balances reasonably, with high recall and economic strong performance that has manageable infrastructure requirements.

8.3 Cost-Aware Optimization and Business Impact

The explicit misclassification cost and threshold optimisation are the. core of aligning the system to the business goals. Although numerous researches have been conducted, most of them argue that these researches are diverse and random. Works are associated with a gain in AUC or F1-score, but not necessarily are associated with real-world savings. Instead, the suggested cost-saving model provides an alternative direct trade off business stakeholders to trade off fraud losses against investigation workload. The recorded decrease in the overall cost compared to naive thresholding values the importance of putting into consideration the economics in model calibration.

8.4 Limitations

One should take into consideration several limitations:

- **Synthetic:** PaySim is a synthetic dataset although realistic. Real-world deployments to be validated against proprietary transaction logs. institution-specific characteristics.
- **Cold-start issue:** New accounts with less history might have invalid history. graph and behavioral characteristics, which may lower the level of detection. Hybridization including heuristics based on the rules or expedited adaptation can be needed.
- **Construction overhead build:** Construct and maintain transactions. Large scale graphs may be a resource-consuming endeavor, but the one-time or this is counteracted by the batched nature of feature computation.
- **Privacy and governance:** Graphs of user relationships increase privacy and compliance issues and have to be dealt with through governing structures and, possibly, privacy-saving methods.

9 Conclusion and Future Work

The paper introduces an AI solution a cost-effective system of payment fraud detection, and it is an XGBoost-based system that has a graph-based feature

engineering system and a cost optimization analysis framework. Using PaySim data, the system has been applied has a high overall recall of 93.67 at a precision of 66.86 and an AUC-ROC of 0.9841, that includes significant additions to the anticipated financial loss below a realistic cost model.

The findings suggest the use of hybrid methods or a mixture of powerful tabular. Among its approaches, explicit cost-sensitive calibration models that are graph-inspired can both perform competitively with state-of-the-art XGBoost and deep learning, and they can accomplish this at low latency and complexity of operation. This makes such means highly fit in real time payment setting.

The future will have various directions that will be taken. Other Research Directions. Part of these extensions are worth further research. Privacy saving and federated learning paradigms, such as privacy aware learning, are increasingly being applied in financial fraud detection systems, where data sharing is an inhibiting consideration that must be of the first priority as is the case in recent surveys. The other potential way is that one can automatically generate transaction graphs, including learning to generate the best graph schema and feature representation, which has yet to be thoroughly studied in the field of graph-based fraud detection. Concept drift handling and online learning are also adaptive algorithms that can be incorporated in model robustness to enhance it robust training strategies. Finally, a non-transactions data modality can also be incorporated to promote the detection of fraud in a complex situation performance.

The suggested architecture and methodology offer a viable roadmap to the use of economic efficient and graph aware, fraud detection systems in contemporary financial contexts.

References

1. Hernandez Aros, L., et al.: Financial fraud detection through the application of machine learning techniques: A literature review. *Humanities and Social Sciences Communications* **11**(1), 1091 (2024)
2. Lucas, Y., et al.: Credit card fraud detection using machine learning: A survey. *arXiv preprint* arXiv:2010.06479 (2020)
3. Phua, C., et al.: A comprehensive survey of data mining-based fraud detection research. *arXiv preprint* arXiv:1009.6119 (2010)
4. Tong, G., et al.: Financial transaction fraud detector based on imbalanced classification optimization. *Applied Soft Computing* **149**, 110984 (2023)
5. Qayoom, A., et al.: A novel approach for credit card fraud transaction detection using deep reinforcement learning scheme. *PeerJ Computer Science* **10**, e1998 (2024)
6. Tayebi, M., El Kafhali, S.: A novel approach based on XGBoost classifier and Bayesian optimization for credit card fraud detection. *Cyber Security and Applications* **3**, 100093 (2025)
7. Bayirr, E.: Fraud detection with CatBoost, XGBoost, LightGBM. Kaggle (2024)
8. Zhou, J., et al.: Graph neural networks: A review of methods and applications. *AI Open* **1**, 57–81 (2020)
9. Longa, A., et al.: Graph neural networks for temporal graphs: State of the art, open challenges, and opportunities. *Transactions on Machine Learning Research* (2023)

10. Adegbola, I.: Graph-augmented RNN models for detecting coordinated fraud rings in transaction networks. SSRN 5319335 (2025)
11. Zou, Y., Cheng, D.: Effective high-order graph representation learning for credit card fraud detection. *arXiv preprint* arXiv:2503.01556 (2025)
12. Rossi, E., et al.: Temporal graph networks for deep learning on dynamic graphs. In: *ICML Workshop on Graph Representation Learning* (2020)
13. Xu, D., et al.: Inductive representation learning on temporal graphs. In: *International Conference on Learning Representations* (2020)
14. Kronberg, S.: Concept drift detection in document classification: An evaluation of ADWIN, KSWIN, and Page-Hinkley using different observation variables. Master's thesis, Umeå University (2024)
15. River ML: PageHinkley—River documentation. `https://riverml.xyz/dev/api/drift/PageHinkley/` (2022)
16. Coralogix AI Blog: 8 concept drift detection methods to use with ML models. `https://coralogix.com/ai-blog/concept-drift-8-detection-methods/` (2025)
17. Optiblack Insights: Explainable AI in SaaS: Financial sector case studies (2025)
18. CFA Institute Research & Policy Center: Explainable AI in finance (2025)